UNIT 1
PROBLEM SOLVING AND ALGORITHM DESIGNING

10TH COMPUTER SCIENCE SINDH TEXTBOOK BOARD

HTTPS://AWINASHGOSWAMI.GITHUB.IO

AGENDA

- Problem Solving
- Algorithm
- Flowchart
- Data Structure

PROBLEM SOLVING

- **Problem solving is the process of finding solutions to an issue**.
- A computer is **not intelligent**—it cannot analyze problems on its own. A **human (programmer)** must:
  - Analyze the problem.
  - Develop instructions.
  - Make the computer carry them out.
- **What is a problem? A problem is a situation that prevents something from being achieved.**

PROBLEM SOLVING PROCESS (4 BASIC STEPS):

DEFINE THE PROBLEM

- This is the **first, most difficult, and most important** step in problem solving.
- It involves **diagnosing the situation** so you focus on the **real problem**.
- You must clearly **describe the problem** in words.
- A well-described problem helps **others** understand it too.

GENERATE ALTERNATIVE SOLUTIONS

- For any problem, there are **more solutions** than the first one you think of.
- **Don't rush** – postpone selecting one solution until you have several alternatives.
- Considering multiple options **improves the quality** of your final solution.
- Make a **list of all feasible solutions**, then evaluate and choose the best one.
- Use both **individual thinking** and **team problem-solving** techniques.

EVALUATE AND SELECT AN ALTERNATIVE

- Generate **many alternatives** before final evaluation.
- **D**on't settle for the **first acceptable** one; it may not be the best.
- If you focus only on getting quick results, you may **miss chances to learn** something new, which is essential for real improvement in problem solving.
- Take time to **compare all options** carefully, then select the most suitable one.

IMPLEMENT AND FOLLOW UP ON THE SOLUTION

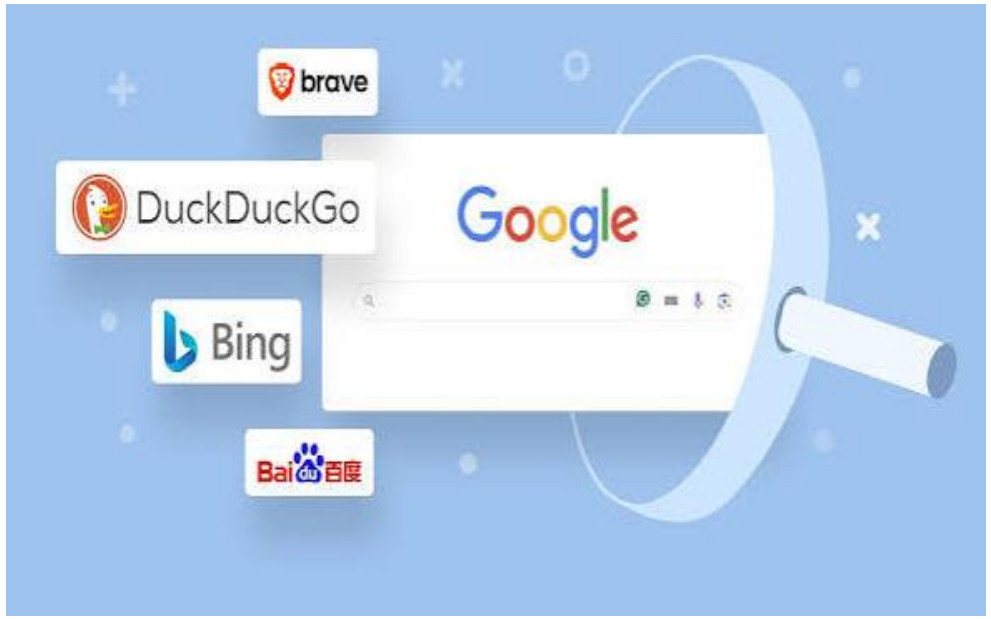
- The plan should also include **what to do if something goes wrong** – if the solution doesn't work as expected.
- After implementing the best solution, **track and measure the results** to answer:
  - *Did it work?*
  - *Was it a good solution?*
  - *Did we learn something that could help with other problems?*
- **While choosing the best solution**, consider its **possible impacts**:
  - Will it solve the problem **without creating new problem**?
  - Is it **acceptable** to everyone involved?
  - Is it **within budget** and **achievable in the given time**?

ALGORITHM

- An algorithm is a set of instructions/steps/rules used to solve a problem.
- In our daily life, we solve multiple problems with step by step such as; making tea, taking direction, completing our study task etc. These can be also taken as algorithm.
- It is a tool for solving a well-specified computational problem.
- In computer problem, algorithm designs can be expressed via pseudocode and flowcharts.

ALGORITHM EXAMPLES IN COMPUTER SCIENCE:

- **Search engines** – take keywords as input, search the database, and return relevant web pages.
- **Online ads** – take age, gender, region, and interests as input, then show ads only to matching users.



OTHER ALGORITHM EXAMPLES: SUM OF TWO NUMBERS

Step 1: Start

Step 2: Declare variables num1, num2 and sum.

Step 3: Read values num1 and num2.

Step 4: Add num1 and num2 and assign the result to sum. sum = num1 + num2

Step 5: Display sum

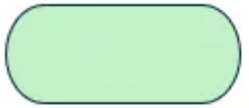
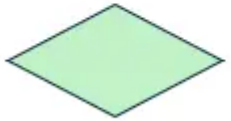
Step 6: Stop

OTHER ALGORITHM EXAMPLES: VOLUME OF A BOX

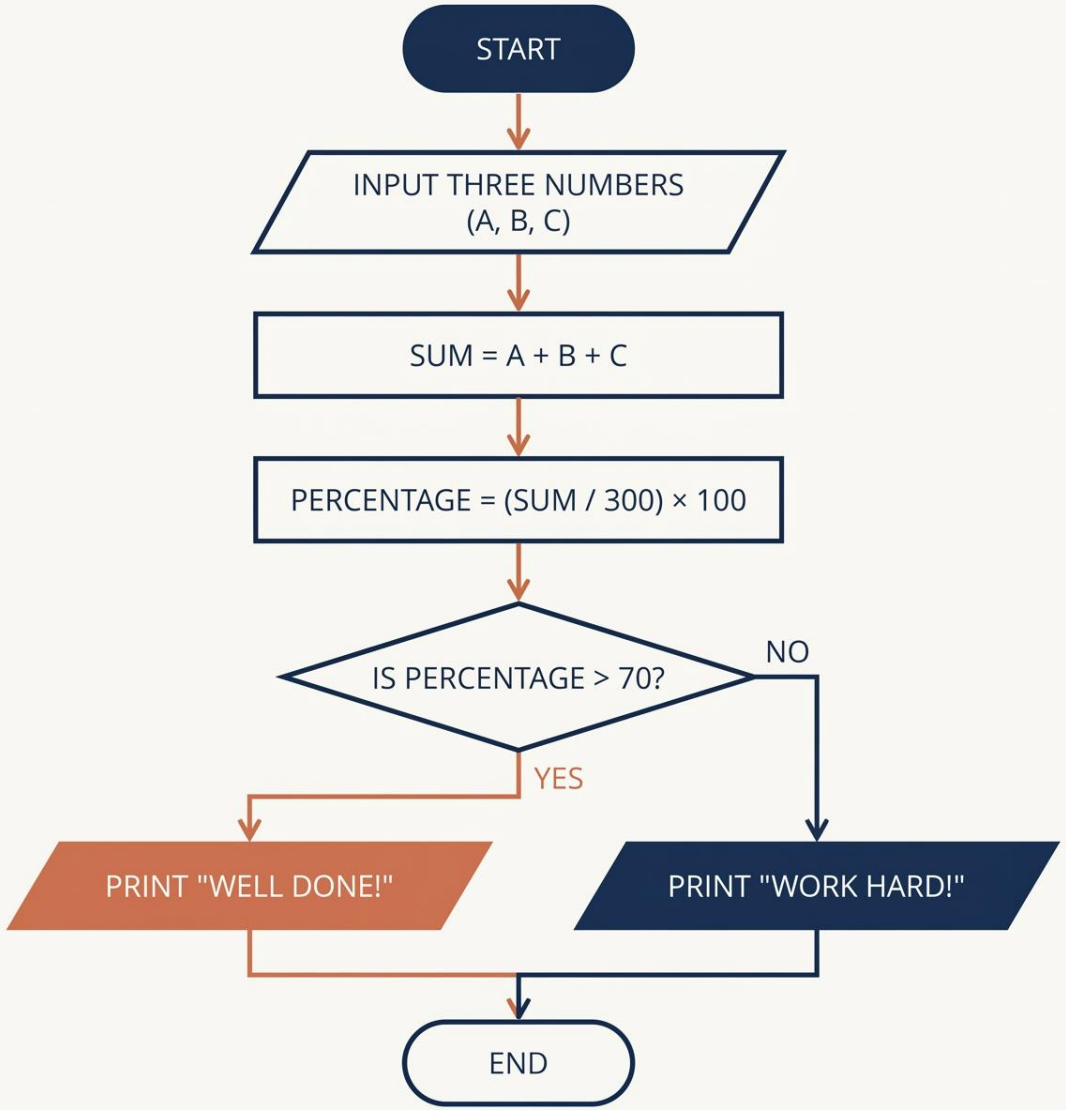
- **Step 1:** Start
- **Step 2:** Declare variables length, width, height and volume.
- **Step 3:** Read values length, width and height.
- **Step 4:** Apply formula. volume = length x width x height
- **Step 5:** Display volume
- **Step 6:** Stop

FLOWCHART

- A flowchart is a visual representation of a step-by-step process or algorithm.
- Flowchart is made up of different symbols to represent program and its flow.

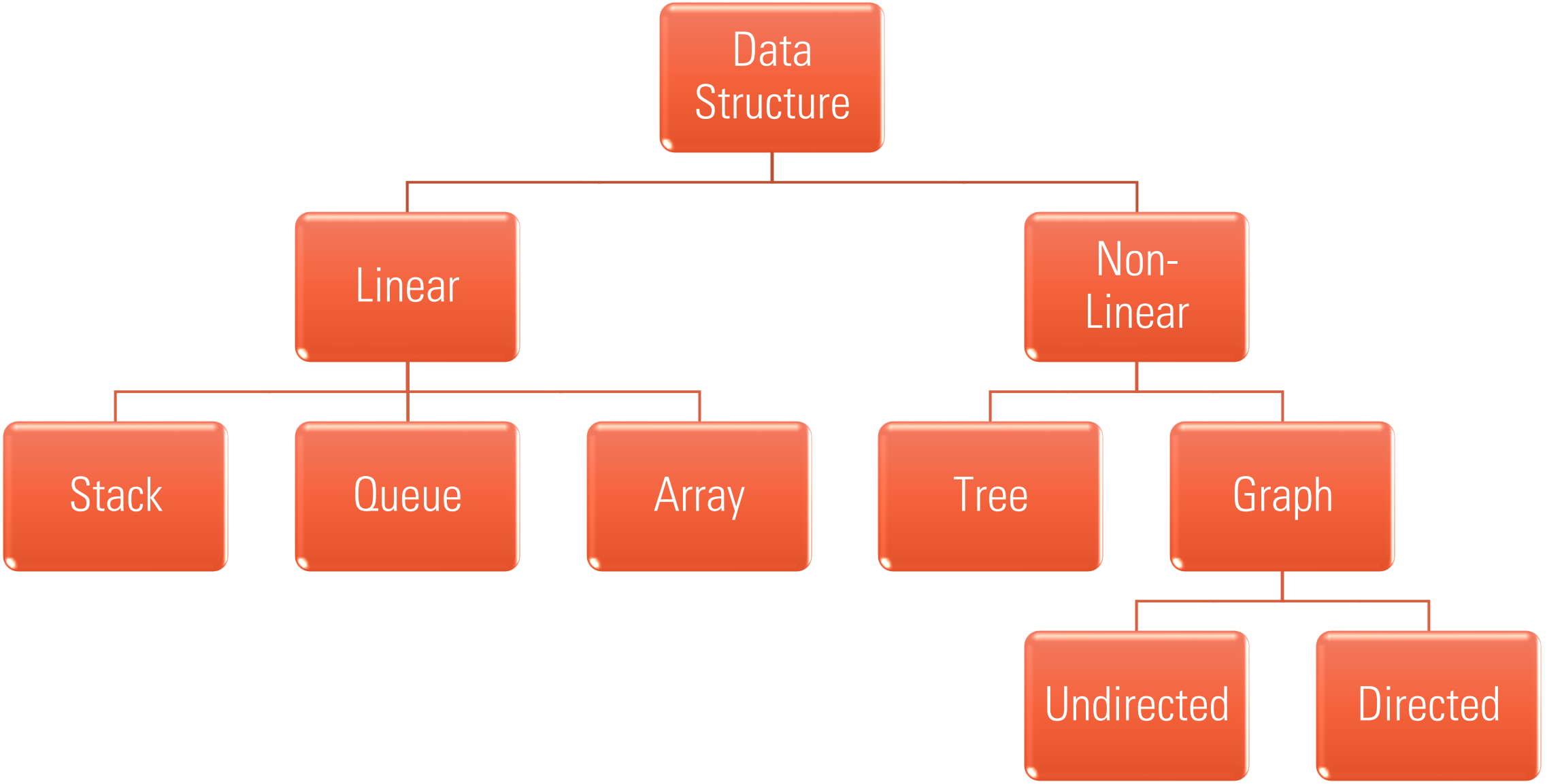
Symbol	Name	Function
	Oval	Represents the start or end of a process
	Rectangle	Denotes a process or operation step
	Arrow	Indicates the flow between steps
	Diamond	Signifies a point requiring a yes/no
	Parallelogram	Used for input or output operations

THREE NUMBER SUM & PERCENTAGE FLOWCHART



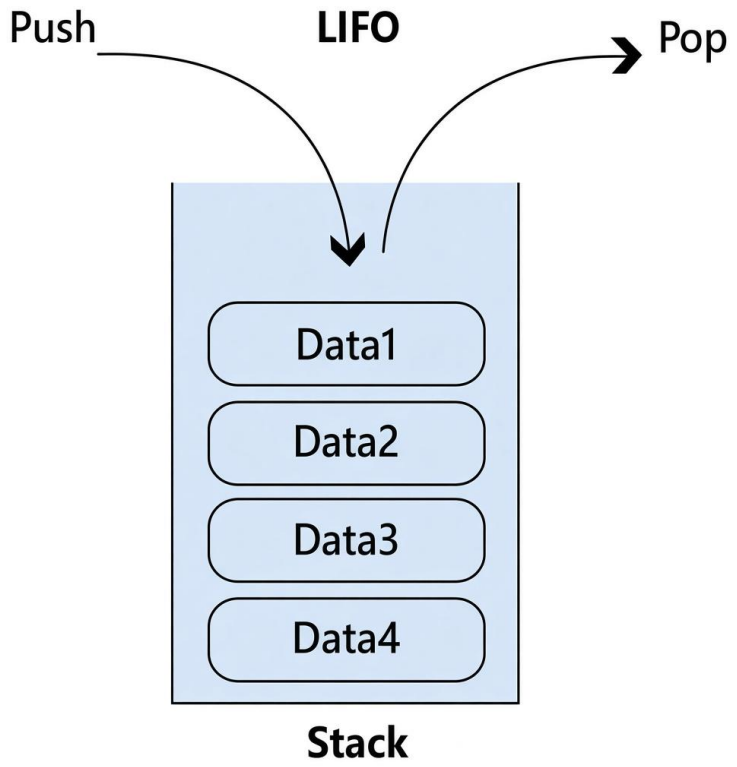
DATA STRUCTURE

- A data structure is particular way of organizing data in a computer to use data effectively.
- Data structure can be either linear or non-linear.
- Linear Data Structure are sequential and inefficient for memory utilization. They are easy to implement. Example of linear data structures are Stack, Queue, Array etc.
- Non-Linear Data Structure are non-sequential but memory efficient. they are difficult to implement. Examples of non-linear data structures are Trees, Graphs etc.


Data Structure
Linear
Non-Linear
Stack
Queue
Array
Tree
Graph
Undirected
Directed

DATA STRUCTURE – LINEAR: STACK

- A Stack is a linear data structure in which items can be added or removed only at the top of stack.
- The order may be LIFO (Last In First Out) or FILO (First In Last Out).
- Examples of a stack are plates that are stacked or put over one another in the canteen.
- The term push is used to insert a new element into the stack and pop is used to remove an element from the stack.
- Stack is in overflow state when it is completely full and we cannot add an item.
- Stack is in underflow state when it is completely empty and we cannot remove an item from it.
- Example of Stack in Computer: Browser Back Button, Undo/Redo

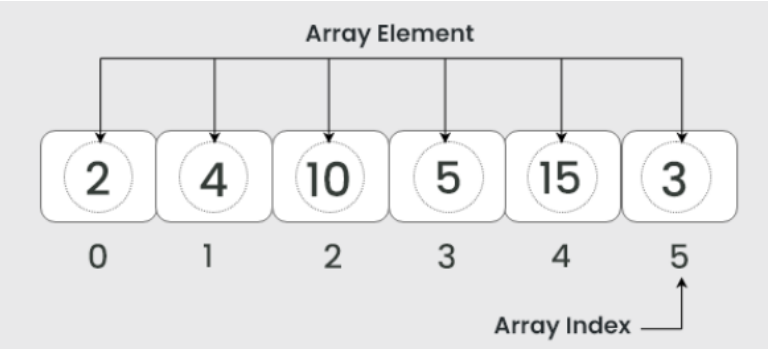


DATA STRUCTURE – LINEAR: QUEUE

- A Queue is a linear data structure in which operations are performed in FIFO (First In First Out) method, which means that element inserted first will be removed first.
- Example of a queue is any queue of students in school.
- Deletions take place at one end called front/head and insertions take place only at the other end called rear/tail.
- Once a new element is inserted into the Queue, it cannot be removed until all the elements inserted before it in the queue are removed.
- The process to add an element into queue is called Enqueue and the process to remove an element from queue is called Dequeue.
- Example of Stack in Computer: Printer Spooler, Keyboard Buffer.

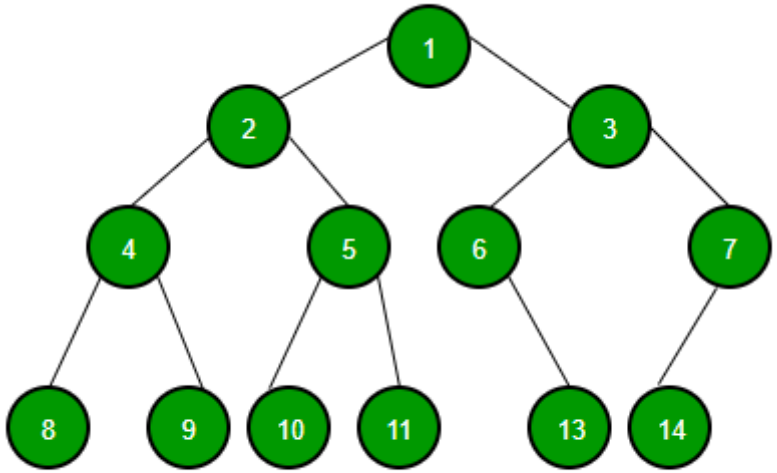
DATA STRUCTURE – LINEAR: ARRAY

- An array is a **linear data structure**, which holds data elements of same data type.
- Each element (*the item stored in the array*) of array can be accessed by an index number (*location of the element in the array*).
- Indexes in an array are consecutive numbers.
- An index of array starts from zero.
- Operations like Traversal, Search, Insertion, Deletion and Sorting can be performed on arrays.
- Example of Array in Computer: Student List, Employee Record.



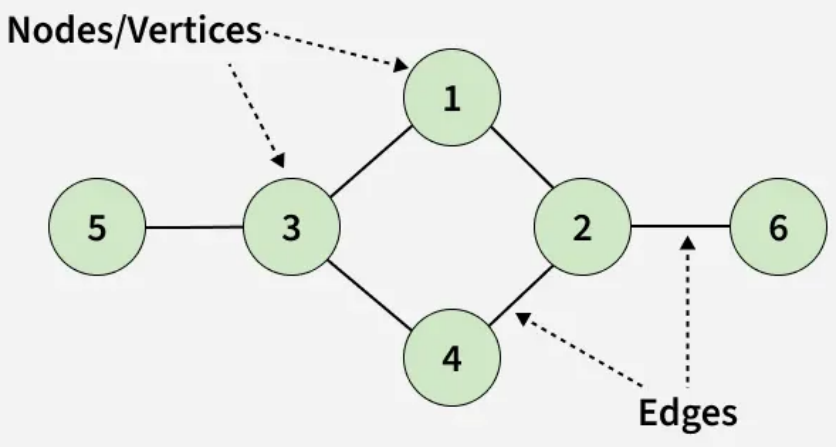
DATA STRUCTURE – NON-LINEAR: TREE

- A tree is a non-linear data structure which is used to represent data containing a hierarchical relationship between elements.
- Tree elements are known as the nodes connected to each other by edges. A root node in tree is a first node.
- In a tree, connected nodes form a parent-child relationship.
- A binary tree is a special type of tree where each node can have at most two children. Some nodes may have one child, two children, or none at all.
- Example of Tree in Computer: Organization of folder and files in OS.



DATA STRUCTURE – NON-LINEAR: GRAPH

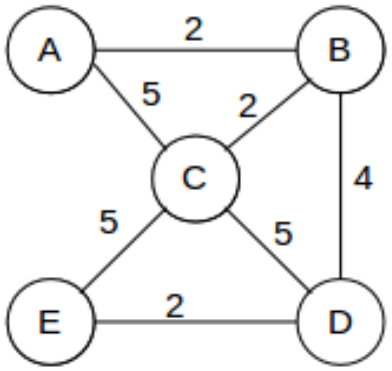
- A graph is a **non-linear data structure** consisting of data elements called nodes/vertices and edges are the lines that connect any two nodes in that graph.
- Each node can contain information like roll number, name of student, marks, etc.
- In graph each node can have any number of edges, there is no any node called root or child.
- Graphs are used to solve network problems.
- Examples of networks include telephone networks, social networks like Facebook, etc.



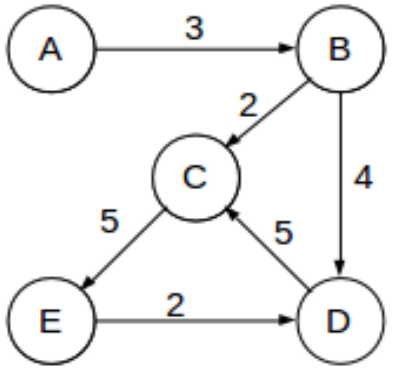
DATA STRUCTURE – NON-LINEAR: GRAPH CONT….

- **Types of graphs:** There are two types of graphs.
  - **Undirected Graph:**
    In an undirected graph, nodes are connected by edges that are all bidirectional.

  - **Directed Graph:**
    In a directed graph, nodes are connected by directed edges – they only go in one direction.



Undirected



Directed